How `.filter()` Works in the Delete Function

This note explains how `Array.prototype.filter()` powers the `deleteExpense` function in the Expense Tracker app.

The line in question

```
setExpenses((prevExpenses) => prevExpenses.filter((e) => e.id !== id));
```

What `.filter()` does — the mental model

`Array.prototype.filter()` walks through every element of an array, runs a **test function** (called a *predicate*) on each one, and builds a **new array** containing only the elements where the predicate returned `true`.

Signature:

```
array.filter(predicate: (element) => boolean): newArray
```

- `true` → keep the element.
- `false` → drop the element.

It does **not** modify the original array. It returns a brand-new array.

Applied to the delete flow

Say the user clicks delete on the expense with `id === "b"`:

```
prevExpenses = [
  { id: "a", name: "Coffee", amount: 100, category: "Food" },
  { id: "b", name: "Bus",    amount: 50,  category: "Travel" },
  { id: "c", name: "Book",   amount: 300, category: "Shopping" },
];

id = "b"; // the one to delete
```

`filter` iterates:

Iteration	`e`	`e.id !== id`	Kept?
1	`{ id: "a", ... }`	`"a" !== "b"` → `true`	Yes
2	`{ id: "b", ... }`	`"b" !== "b"` → `false`	No
3	`{ id: "c", ... }`	`"c" !== "b"` → `true`	Yes

Result — a new array with two elements:

```
[
  { id: "a", name: "Coffee", amount: 100, category: "Food" },
  { id: "c", name: "Book",   amount: 300, category: "Shopping" },
]
```

That new array is handed to `setExpenses`, which stores it as the new state and triggers a re-render.

Reading the predicate: `(e) => e.id !== id`

Two `id`s to keep straight:

- `e.id` — the id of the **current expense being checked** in the loop.
- `id` — the parameter of `deleteExpense`, the one the user clicked.

The condition says: **"keep this expense if its id is NOT the one being deleted."** Everything except the target survives.

If you flipped it to `e.id === id`, you'd keep *only* the deleted one — the opposite of what you want.

Why `.filter()` instead of `.splice()` or `delete`?

This is the part that matters for React.

`.splice()` mutates

```
prevExpenses.splice(1, 1); // removes index 1 in place
setExpenses(prevExpenses);
```

This changes the existing array. Since `setExpenses` receives the **same reference** it had before, React's bail-out check (`Object.is(oldState, newState)`) returns `true` and **skips the re-render**. The UI won't update.

`delete` leaves a hole

```
delete prevExpenses[1]; // → [expense, empty, expense]
```

Doesn't remove the slot, just sets it to `undefined`. Length stays the same. Also mutates.

`.filter()` is immutable

Returns a **new array reference**. React sees `oldRef !== newRef`, triggers re-render, and every consumer (`ExpenseList`, `TotalExpense`) gets the updated data. This is the React-friendly way.

Why the functional updater `(prevExpenses) => ...`?

You could also write:

```
setExpenses(expenses.filter((e) => e.id !== id));
```

It would usually work, but the functional form is safer because it guarantees you're filtering the **latest** state, not a stale closure. If two deletes fire quickly (e.g. rapid clicks or batched updates), the closure version might filter an outdated array and lose an update.

Rule of thumb: **when the new state depends on the old state, use the functional updater.**

Bonus — `filter` vs `map` vs `find`

Beginners often mix these up. All three iterate, but they return different things:

Method	Returns	Use when…
`filter`	new array (subset)	you want to keep *some* elements
`map`	new array (same length, transformed)	you want to change *every* element
`find`	single element or `undefined`	you want *one specific* element

So delete = `filter`. Update-one-item (e.g. "edit expense") = `map`. Look up by id = `find`.

Try it in the console

To cement the intuition, paste this into a browser console:

```
const arr = [1, 2, 3, 4, 5];
const evens = arr.filter(n => n % 2 === 0);
console.log(evens); // [2, 4]
console.log(arr);   // [1, 2, 3, 4, 5]  ← original untouched
```

The original stays intact; you get a fresh array back. That immutability is exactly what React needs to detect the change and update the UI.